

Tarea 6

Problema del Logaritmo Discreto

► Areli Alejandra Guevara Badillo

► viernes, 5 de diciembre de 2025

Instrucciones

Diseña e implementa un programa de computadora que utilice fuerza bruta para resolver los ejercicios a continuación. Puedes usar solo uno de los siguientes lenguajes de programación: C/C++, Java, Python o C#.

Desarrollo

```
1 def modFuerzaBruta(a, b, c):
2     limit = b + 1
3
4     for i in range(1, limit):
5         if pow(a, i, b) == c:
6             return i
7
8     return None
```

La función recibe tres números:

- a (Base): el número que se va a elevar a la potencia x .
- b (Módulo): el número por el que dividimos.
- c (Objetivo): el resultado que estamos buscando.

Definimos un límite de pruebas `limit = b + 1`. Esto ya que, en la aritmética modular, los resultados se repiten en ciclos. Si pruebas tantas veces como el tamaño del módulo (b) y no encuentras la respuesta, jamás la encontrarás. Esto evita un bucle infinito.

En el ciclo `for i in range(1, limit):` se prueban los números uno por uno para la i .

`pow(a, i, b)`: Calcula $a^i \bmod b$.

Al usar esta función nativa de Python, calcula el resultado modular paso a paso. Esto hace que sea rápido y no llene la memoria de la PC.

Si encontramos el número, la función se detiene inmediatamente y te entrega el valor de i .

Si el bucle for termina (probó todos los números hasta el límite) y nunca entró al if, la función llega a esta línea final, devuelve None.

Ejercicios

Por cada uno de las siguientes ecuaciones, encuentra el valor entero más pequeño para $x \geq 1$ que satisfaga la ecuación.

a) $3^x \bmod 641 = 285$

```
Ingrese la base a: 3
Ingrese el modulo b: 641
Ingrese el residuo c: 285

El menor exponente es x = 638
Verificación: 3^638 mod 641 = 285
Tiempo transcurrido: 0.0006 segundos
```

b) $2^x \bmod 24,499 = 2,697$

```
Ingrese la base a: 2
Ingrese el modulo b: 24499
Ingrese el residuo c: 2697

El menor exponente es x = 20000
Verificación: 2^20000 mod 24499 = 2697
Tiempo transcurrido: 0.0061 segundos
```

c) $5^x \bmod 3,558,889,333 = 2,199,600,647$

```
Ingrese la base a: 5
Ingrese el modulo b: 3558889333
Ingrese el residuo c: 2199600647

El menor exponente es x = 1234554321
Verificación: 5^1234554321 mod 3558889333 = 2199600647
Tiempo transcurrido: 6786.9177 segundos
```

d) $2^x \bmod 925,524,359,093 = 107,922,727,226$

```
Ingrese la base a: 2
Ingrese el modulo b: 925524359093
Ingrese el residuo c: 107922727226

El menor exponente es x = 1000
Verificación: 2^1000 mod 925524359093 = 107922727226
Tiempo transcurrido: 0.0012 segundos
```

e) $3^x \bmod 500,000,009 = 187,776,257$

```
Ingrese la base a: 3
Ingrese el modulo b: 500000009
Ingrese el residuo c: 187776257

El menor exponente es x = 500000000
Verificación: 3^500000000 mod 500000009 = 187776257
Tiempo transcurrido: 475.5299 segundos
```

f) $2^x \bmod 12,575,817,766,556,298,347 = 5,508,961,339,840,704,716$

```
Ingrese la base a: 2
Ingrese el modulo b: 12575817766556298347
Ingrese el residuo c: 5508961339840704716

El menor exponente es x = 100000000
Verificación: 2^100000000 mod 12575817766556298347 = 5508961339840704716
Tiempo transcurrido: 843.3579 segundos
```

g) $3^x \bmod 113,750,260,501,994,812,111,267,656,933,598,813,423 = 97,139,203,429,660,076,870,914,437,708,269,649,628$

El programa se ejecutó el martes 02 de diciembre de 2025 a las 19:23 hrs.

A causa de tiempo el programa se detuvo el jueves 04 de diciembre de 2025 a las 18:38 hrs.

Por lo que estuvo corriendo un total de 1 día con 23 hrs y 15 minutos sin llegar a la solución.

Preguntas

Características del ordenador donde se ejecutó el programa:

- ▶ Tamaño de memoria (RAM): 16GB
- ▶ Nombre del procesador: 12th Gen Intel(R) Core(TM) i5-12450H
- ▶ Velocidad del procesador: 2.50 GHz

Averigua el nombre y la complejidad computacional de los algoritmos para resolver el problema del logaritmo discreto. ¿Cuál es el mejor? ¿Por qué? Enumera las referencias donde encuentre los algoritmos y su complejidad computacional.

1. Fuerza Bruta (Búsqueda Exhaustiva)

Es el método más simple e intuitivo. Consiste en probar sistemáticamente cada posible exponente x (desde 0, 1, 2, ... hasta $N-1$) hasta encontrar uno que satisfaga la ecuación $g^x \equiv h \pmod{n}$. No requiere matemáticas avanzadas, solo potencia de cálculo pura.

- **Complejidad:** $O(N)$ (Exponencial).

Para un número de k bits, $N \approx 2^k$. Tienes que hacer 2^k operaciones. Es inviable para claves mayores de 40 bits.

2. Baby-step Giant-step (Pasos de bebé, Pasos de gigante)

Es un algoritmo de tipo "encuentro en el medio" (meet-in-the-middle). Intercambia memoria por velocidad. Divide la búsqueda en dos partes: calcula una tabla de $\approx \sqrt{N}$ valores pequeños ("baby steps") y luego da pasos grandes ("giant steps") buscando coincidencias en esa tabla. Convierte un problema lineal en uno de raíz cuadrada.

- **Complejidad:** $O(\sqrt{N})$

Reduce el tiempo drásticamente, pero requiere $O(\sqrt{N})$ de espacio en memoria, lo cual es su gran limitante.

3. Algoritmo Rho de Pollard (ρ)

Utiliza un enfoque probabilístico basado en "caminatas aleatorias". Genera una secuencia de números que eventualmente debe repetirse (formando un ciclo en forma de la letra ρ). Al encontrar dos números iguales en la secuencia (una colisión), se puede resolver la ecuación para el logaritmo.

- **Complejidad:** $O(\sqrt{N})$ (tiempo esperado).

Tiene la misma velocidad teórica que el Baby-step Giant-step, pero con la ventaja crítica de que **casi no consume memoria** ($O(1)$).

4. Algoritmo Pohlig-Hellman

Este algoritmo no ataca el grupo entero de golpe, sino que usa la factorización del orden del grupo (N). Si N se puede descomponer en factores primos pequeños ($N = p_1^{e_1} \dots p_k^{e_k}$), el algoritmo resuelve el logaritmo para cada factor pequeño y luego combina las respuestas usando el Teorema Chino del Resto.

- **Complejidad:** $O(\sum e_i (\log N + \sqrt{p_i}))$
 - **Caso ideal:** Si N está compuesto solo por primos pequeños ("smooth number"), este algoritmo es extremadamente rápido.
 - **Caso peor:** Si N es un número primo grande, el algoritmo degenera a una complejidad de $O(\sqrt{N})$.

5. Index Calculus (Cálculo de Índices)

A diferencia de los anteriores, este algoritmo es específico para grupos donde se puede definir una "base de factores" (como los números enteros módulo p).

1. Recopila relaciones lineales de logaritmos de primos pequeños.
2. Resuelve un sistema masivo de ecuaciones lineales.

- **Complejidad:** $L_p[1/2]$ (Sub-exponencial).

Es mucho más rápido que la raíz cuadrada para números muy grandes, pero no funciona en curvas elípticas generales.

6. General Number Field Sieve (GNFS)

Es el algoritmo más avanzado y matemáticamente complejo conocido hoy en día para el logaritmo discreto clásico. Utiliza teoría de números algebraica (campos numéricos) para crear relaciones de manera mucho más eficiente que el Index Calculus estándar.

- **Complejidad:** $L_p[1/3, 1.923]$ (Sub-exponencial fuerte).

Es asintóticamente el algoritmo más rápido existente para cuerpos finitos grandes.

Conclusión

Tras analizar las distintas opciones, se determina que el **General Number Field Sieve (GNFS)** es el algoritmo más eficiente y potente que existe actualmente para resolver el Problema del Logaritmo Discreto en los sistemas numéricos clásicos.

Su superioridad técnica radica en su capacidad para manejar grandes cifras. Mientras que los métodos tradicionales requieren un tiempo de cálculo que crece desproporcionadamente al aumentar el tamaño de los números, el **GNFS** utiliza una estructura matemática avanzada que le permite encontrar la solución en un tiempo significativamente menor.

Debido a su alta efectividad, este algoritmo es el que dicta los estándares de seguridad actuales. Es la razón fundamental por la cual la industria tecnológica ha tenido que abandonar el uso de claves de seguridad pequeñas y adoptar claves mucho más extensas (de 2048 bits o más), ya que son las únicas capaces de resistir la potencia de cálculo de este método.

Referencias

- ▶ Registro discreto: algoritmos para programación competitiva. (agosto 19, 2025). <https://cp-algorithms.com/algebra/discrete-log.html>
- ▶ Santamaría, J., & Sadornil, D. (2013). El logaritmo discreto y sus aplicaciones en criptografía [Tesis de Licenciatura, Universidad de Cantabria]. <https://repositorio.unican.es/xmlui/bitstream/handle/10902/3101/Jennifer%20Santamaria%20Fernandez.pdf>
- ▶ Padmavathy, R., & Bhagvati, C. (2011). Discrete logarithm problem using index calculus method. *Mathematical and Computer Modelling*, 55(1–2), 161–169. <https://doi.org/10.1016/j.mcm.2011.02.022>
- ▶ colaboradores de Wikipedia. (2024, January 29). Algoritmo rho de Pollard (logaritmos discretos). Retrieved from [https://es.wikipedia.org/wiki/Algoritmo_rho_de_Pollard_\(logaritmos_discretos\)](https://es.wikipedia.org/wiki/Algoritmo_rho_de_Pollard_(logaritmos_discretos))
- ▶ Rodríguez Chuquimango, S. P. (2018). *Teoría de la CURVAS elípticas sobre campos finitos y su aplicación en la solución del problema de logaritmo discreto*. (pág. 52-58, 59-66). https://alicia.concytec.gob.pe/vufind/Record/UNAC_4fee431c80c93d0c9e23411485eff98f
- ▶ Cabrero del Campo, S. (2025). El problema del logaritmo discreto en curvas elípticas. (pág. 49-53). <https://uvadoc.uva.es/handle/10324/78412>